

Week 1 Deliverable Report

User Management Mini API — How It Was Built & How to Test

AI Engineer Internship Program

September 2026

1. Introduction

This report documents the **Week 1** deliverable of the AI Engineer Internship Program (Phase 1 — Technology Foundations).

Objective: Build a simple CRUD mini API for user management using **Node.js + TypeScript + PostgreSQL**, with complete authentication, packaged and runnable via **Docker Compose**.

Project folder: `week1-user-api/`

2. Requirements Covered

Requirement	Implementation
REST API with Express.js	Express + TypeScript layered architecture
TypeScript basics	Types, interfaces, Zod schemas
PostgreSQL + ORM	Prisma ORM with migrations
Auth / Authorization	JWT + roles USER / ADMIN
Validation & error handling	Zod + centralized error middleware
Logging	Structured JSON logger
Docker & Docker Compose	Dockerfile + docker-compose.yml
Extra (demo-ready)	Swagger UI + admin seed script

3. Tech Stack

- **Runtime:** Node.js 20
- **Framework:** Express.js
- **Language:** TypeScript
- **Database:** PostgreSQL 16
- **ORM:** Prisma
- **Auth:** JWT (`jsonwebtoken`) + bcrypt password hashing
- **Validation:** Zod
- **API Docs:** Swagger UI (`/api/docs`)

- **Containers:** Docker + Docker Compose
-

4. How It Was Built

4.1 Project architecture

Layered Express structure:

Client → Routes → Controllers → Services → Prisma → PostgreSQL

Folder layout:

```
week1-user-api/  
  src/  
    config/          # env, swagger  
    middlewares/     # auth, validate, errors  
    modules/  
      auth/          # register, login  
      users/         # CRUD  
    lib/             # prisma, logger, AppError  
    app.ts  
    server.ts  
  prisma/  
    schema.prisma  
    migrations/  
    seed.ts  
  docker-compose.yml  
  Dockerfile  
  README.md
```

4.2 Database design

Model: User

Field	Type	Notes
id	UUID	Primary key
email	String	Unique
password	String	bcrypt hash (never returned in API)
name	String	Display name
role	Enum	USER or ADMIN (default USER)
createdAt / updatedAt	DateTime	Auto-managed

Migrations are stored under `prisma/migrations/` and applied automatically when the API container starts.

4.3 Authentication & roles

1. **Register** (POST `/api/auth/register`) creates a user with role **USER**.
2. **Login** (POST `/api/auth/login`) returns a JWT access token.

3. Protected routes require header: `Authorization: Bearer <token>`.
4. Role rules:
 - **USER:** manage own profile (`/me`, get/update self)
 - **ADMIN:** list all users, change roles, delete users

4.4 API endpoints

Method	Path	Access
GET	/	Public — API index
GET	/health	Public — health check
GET	/api/docs	Public — Swagger UI
POST	/api/auth/register	Public
POST	/api/auth/login	Public
GET	/api/users/me	Authenticated
GET	/api/users	Admin only
GET	/api/users/:id	Self or Admin
PATCH	/api/users/:id	Self or Admin (role change: Admin only)
DELETE	/api/users/:id	Admin only

4.5 Docker packaging

`docker-compose.yml` runs two services:

1. **db** — PostgreSQL 16 (host port **5433** → container 5432)
2. **api** — builds the Node app, runs migrations, seeds admin, starts server on port **3000**

On container start (`docker-entrypoint.sh`):

1. `prisma migrate deploy`
2. `prisma db seed`
3. `node dist/server.js`

Seeded admin account

Email	Password	Role
admin@example.com	Admin123!	ADMIN

Note: Port **5433** is used on the host to avoid conflict with a local PostgreSQL already using 5432. Inside Compose, the API connects to `db:5432`.

5. How to Run

5.1 Full stack with Docker (recommended for mentor demo)

```
cd week1-user-api
docker compose up --build
```

Then open:

- API index: <http://localhost:3000>
- Health: <http://localhost:3000/health>
- Swagger: <http://localhost:3000/api/docs>

Stop everything:

```
docker compose down
```

5.2 Local development (optional)

```
cp .env.example .env
docker compose up db -d
npm install
npx prisma migrate dev
npx prisma db seed
npm run dev
```

6. How to Test Everything (Mentor Checklist)

Use **Swagger** at <http://localhost:3000/api/docs> for the full walkthrough.

Test 1 — System is up

1. Open <http://localhost:3000/health>
2. Expect: `{"status": "ok"}`
3. Open <http://localhost:3000> → see API index with links

Test 2 — Admin login

1. In Swagger, run **POST /api/auth/login**
2. Body:

```
{
  "email": "admin@example.com",
  "password": "Admin123!"
}
```

3. Expect 200 with `user.role = "ADMIN"` and `accessToken`
4. Click **Authorize** → paste the token → **Authorize**

Test 3 — Register a new user

1. Run **POST /api/auth/register** with a **new** email:

```
{  
  "email": "demo.user@example.com",  
  "password": "User1234!",  
  "name": "Demo User"  
}
```

2. Expect 201 and `role = "USER"`

3. If email already exists → 409 Email already registered (correct behavior)

Test 4 — Admin can list users

1. Still authorized as **admin**

2. Run `GET /api/users`

3. Expect 200 and an array of users

Test 5 — Current profile

1. Run `GET /api/users/me`

2. Expect your logged-in user profile

Test 6 — User cannot list all users (RBAC)

1. Logout / clear Authorize

2. Login as the new user (`demo.user@example.com`)

3. Authorize with the new token

4. Run `GET /api/users`

5. Expect 403 **Insufficient permissions**

Test 7 — Self update

1. As the user, run `PATCH /api/users/{id}` with that user's own id

2. Body example:

```
{  
  "name": "Demo User Updated"  
}
```

3. Expect 200 with updated name

Test 8 — User cannot change role

1. As the user, try:

```
{  
  "role": "ADMIN"  
}
```

2. Expect 403 Only admins can change roles

Test 9 — Admin delete

1. Login again as admin and Authorize
2. Run **DELETE** `/api/users/{id}` for a non-admin test user
3. Expect 204 (no body)

Test 10 — Validation

1. Call register with invalid email or short password
 2. Expect 400 **Validation failed** with field errors
-

7. Quick curl Examples

Health

```
curl http://localhost:3000/health
```

Admin login

```
curl -s -X POST http://localhost:3000/api/auth/login \  
  -H 'Content-Type: application/json' \  
  -d '{"email":"admin@example.com","password":"Admin123!"}'
```

List users (replace TOKEN)

```
curl http://localhost:3000/api/users \  
  -H "Authorization: Bearer TOKEN"
```

8. What to Say in the Demo (30 seconds)

This is the Week 1 mini API: Express + TypeScript + Prisma + PostgreSQL.
It supports JWT authentication with USER and ADMIN roles, validated user CRUD, Swagger documentation, a seeded admin account, and runs end-to-end with Docker Compose.

9. Conclusion

The Week 1 deliverable meets the internship roadmap requirements:

- CRUD user management API
- Complete JWT auth + role-based access
- PostgreSQL with Prisma migrations
- Validation, error handling, and logging
- Packaged and runnable via Docker Compose
- Swagger + seed for fast mentor review

Primary demo entry points:

`docker compose up --build` → <http://localhost:3000/api/docs>